

Lists

Jarmo Hurri

Intro

- consider the following programs
 - shopping: adding orders, prioritising, removing, traversing
 - social: keeping a record of previous calls (privacy: remove)
 - computing: keeping track of processes running on computer
 - in all cases, values need to be
 - stored
 - ordered
 - possibly removed
 - possible reordered
- can be solved using an *abstract data type*: list
 - one of the most practical data types

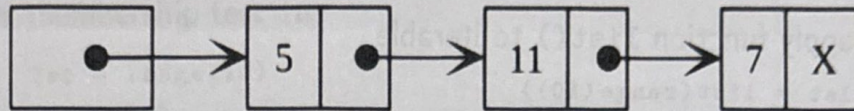
Example 1

```
lst = [5, 11, 7] # bind lst to list with 3 elements
for e in lst: print(e) # traverse, printing each element
print(lst[0], lst[1], lst[2]) # access with 0-based indices
lst[2] *= 5 # modify element
print(lst[-1]) # access last element
print(len(lst)) # number of elements
print(lst[4]) # indexing error
```

List

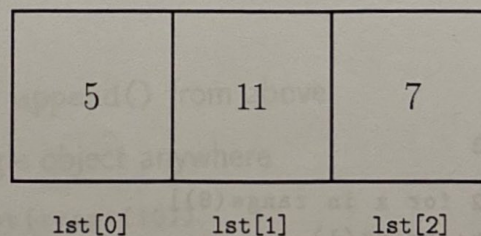
- *abstract data type* (ADT)
 - specifies operations supported
 - does not specify how these are implemented
- this one supports (among other things)
 - construction (in a number of ways)
 - *traversing* (going through all/some elements)
 - searching elements
 - modifying elements
 - adding elements (anywhere)
 - removing elements (anywhere)
 - reordering
- *dynamic*: flexible size
- two common ways to implement lists
 - linked nodes
 - contiguous block of memory

List as linked nodes



- example list of integers with elements {5, 11, 7}
- *link* from *node* to next node
- empty start node
- last node has no link to next
- we will implement this later

List as contiguous memory block



- data store in block of memory
- memory reallocated if runs out of space
- this is how Python does it
- accessing and modifying elements is *fast*
- adding / removing can be slow

Constructing lists

- provide elements explicitly

```
lst = ['wait', 'what']
```

- apply function `list()` to iterable

```
lst = list(range(10))
```

- build gradually by appending values

```
lst = []  
for x in range(10): lst.append(x**2)
```

- use *list comprehension*

```
[expr for x in iterable if cond]
```

1. take values from iterable
2. filter ones matching cond (optional)
3. apply expr to remaining ones

```
lst = [f'val {x}' for x in range(10) if x % 3 == 0]
```

Traversing lists

- list is iterable

```
lst = [x % 2 for x in range(8)]  
for l in lst: print(l)
```

- elements can be accessed with 0-based *index*

```
lst = [c for c in 'thingamabob'] # string is also iterable  
for i in range(len(lst)):  
    if i % 2 == 0: print(lst[i])
```

Searching lists

- membership test in

```
lst = range(10)
10 in lst
```

- method `count()` returns number of occurrences
- method `index()` finds first index of match (`-1` if not found)

```
lst = [10 - abs(x-3) for x in range(7)]
print(lst)
val = 8
print(lst.count(val))
print(lst.index(val))
```

Adding elements

- remember `append()` from above
- insert single object anywhere

```
lst = list(range(10))
lst.insert(0, 55) # add 55 before element at index 0
print(lst)
```

- append all from another iterable

```
lst = list(range(10))
lst.extend(range(5))
print(lst)
```

Removing elements

- remove from end and return

```
lst = list(range(10))
print(lst.pop())
print(lst)
```

- remove from any index and return

```
lst = [10 - x for x in range(10)] # 10, 9, ..., 1
print(lst.pop(3))
print(lst)
```

- remove 1st occurrence of element

```
lst = [abs(x - 5) for x in range(10)]
print(lst)
lst.remove(3)
print(lst)
```

Reordering

- reverse

```
lst = list(range(10))
for l in reversed(lst): print(l, end=' ') # reverse iterator
print(lst)
lst.reverse() # reverse in-place
print(lst)
```

- sort (ascending or descending order)

```
lst = [8, -1, 5]
print(sorted(lst)) # returns new sorted list
print(lst)
lst.sort(reverse=True) # sorts in-place, descending
print(lst)
```

- randomize order

```
import random
lst = list(range(10))
# take len(lst) elements in random order
print(random.sample(lst, len(lst)))
```

Some operations on lists in Python (for list lst)

	method	description
长度	$\leftarrow \text{len}(\text{lst})$	return number of elements
结尾加一	$\leftarrow \text{lst.append}(\text{o})$	add o to back
指定位置加	$\leftarrow \text{lst.insert}(\text{index}, \text{o})$	add to 0-based index
删位置	$\leftarrow \text{lst.pop}(\text{index}=-1)$	remove and return (default end)
删指定	$\leftarrow \text{lst.remove}(\text{o})$	remove 1st occurrence of o)
结尾加多	$\leftarrow \text{lst.extend}(\text{it})$	add elements of it at end
T/F	$\leftarrow \text{x in lst}$	test whether x is element
找'o'位置	$\leftarrow \text{lst.index}(\text{o})$	return 1st index of o
多少个'o'	$\leftarrow \text{lst.count}(\text{o})$	return count of o s
大小排序	$\leftarrow \text{lst.sort}()$	sort in-place
随机取n	$\leftarrow \text{random.sample}(\text{lst}, \text{n})$	return list with n random elements

Example 2

1. Write a program which reads nonnegative integers from the user and places them in a list using the methods discussed above. User signals end of input by typing a negative integer. Before exiting, the program prints the list.
2. Modify your program so that at all times, the list is ordered in ascending (increasing) order.

Example 3

Write a program which takes as input a list of strings, and removes those strings whose length is at most 4 characters. Use

1. a while-loop
2. list comprehension.

Tuple

- *tuple* is container of fixed length
- for example $\leftrightarrow$

```
t = (2, 'text') # tuple with two elements
print(t)
print(len(t))
(num, word) = t # unpacking elements
print(num, word)
```

- convenient data type for some programming tasks
- tuple with two elements is called *pair*

Automatic access to index with enumerate()

- `enumerate(lst)` returns iterable of pairs `(i,el)` where `el` is element of `lst` and `i` is index of `el` in `lst`
- example

```
lst = ['let', 'us', 'see']
for p in enumerate(lst): print(p) # traverse pairs
for (i, w) in enumerate(lst): print(i, w) # unpack
```

Processing parallel iterables with zip()

- consider following product inventory data
- | | | | | | |
|------------|------|----|------|-----|-----|
| quantity | 34 | 5 | 19 | 18 | 82 |
| unit price | 81.3 | 44 | 71.3 | 113 | 8.4 |
- can be stored in two *parallel arrays* `quantity` and `unit_price`
 - say we want to process each column of table
 - `zip()` provides way to do this ↪

```
quantity = [34, 5, 19, 18, 82]
unit_price = [81.3, 44, 71.3, 113, 8.4]

# traverse columns
for p in zip(quantity, unit_price): print(p)

# calculate total price
total_price = 0
for (q, p) in zip(quantity, unit_price):
    total_price += q * p
print(total_price)
```

Slices

- slices provide way to retrieve / assign sublist
- example

```
lst = list(reversed(range(10)))  
print(lst)  
print(lst[3:6]) # sublist at indices 3,4,5 (6 excluded)
```

- in basic case `lst[b:e]`, length of sublist is $(e-b)$
- if `b` is excluded, start from beginning of list
- if `e` is excluded, continue until end of list

Example 4

Consider the following list generation:

```
import random  
n = 12  
lst = random.sample(list(range(n)), n)
```

We want to find a list of the differences between consecutive elements. An example is shown below:

```
list: [10, 4, 11, 2, 0, 8, 1, 6, 3, 5, 7, 9]  
diff: [-6, 7, -9, -2, 8, -7, 5, -3, 2, 2, 2]
```

Use list comprehensions, slices and `zip()` to write a one-line program that accomplishes this task.

Note that for lists `a` and `b` of different lengths, `zip(a,b)` will stop at the end of the shorter list.

Homework 1

Starting from an empty list, write down list contents after each of the following steps, executed in the order given.

1. append 1
2. append 6
3. insert value 0 at index 1
4. delete element with value 6
5. remove element at index 0

Homework 2

Write a program which reads integers from the user, stores the negative integers into one list, positive integers into another list, and prints both of these lists when the user inputs 0.

Homework 3

Use `append()` or list comprehensions (your choice) to create the following lists:

1. `[0, 1, 4, 9, 16, 25, 36, 49]`
2. `['2**0 is 1', '2**1 is 2', '2**2 is 4', '2**3 is 8']`
3. `[(0, 5), (1, 6), (2, 7), (3, 8), (4, 9), (5, 10)]`

Homework 4

Write a program which executes the two stages below:

1. during the first stage reads individual words from the user
 - user indicates end of this stage by typing single-character word
`!`
2. during the second stage reads more words from the user,
printing
`hit`
exactly when the word was seen in the first stage.
 - again, user indicates end of this stage by typing
single-character word `!`

Homework 5

Write a program which first reads nonnegative integers from the user (negative value stops reading), and then prints two rows:

- list of all integers
- row with each seen value printed exactly once.

Sample output:

```
[6, 3, 8, 3, 3, 6, 6, 8, 9]
```

```
6 3 8 9
```

Homework 6

The following program code creates a list of random lists:

```
import random
n = 10
list_of_lists = [random.sample(list(range(n)), n) for _ in range(5)]
```

Write a program which *flattens* this list of lists: the program creates one list containing all elements from all lists, in the order they are shown in `print(list_of_lists)`.

Homework 7

Write a program, which

1. reads nonnegative integers from the user
2. stores each integer in a list
 - 2.1 if the integer is not in the list, it is added to the front of the list
 - 2.2 if the integer is in the list, it is moved to the front of the list.

When the user inputs a negative integer, the program prints the list.

This type of an algorithm is useful when precedence is given to most recently seen items (e.g. recording most recent accesses in security control, caching most recent web pages).

Homework 8

Write a program which executes the three stages below:

1. reads words from the user into a list
 - end of stage indicated by word !
2. reads indices from the user
 - end of stage indicated by a negative integer
 - you may assume that each index is input at most once
3.
 - 3.1 prints original word list
 - 3.2 prints list of indices
 - 3.3 prints word list with words at given indices removed.

Sample output:

```
original: ['cat', 'is', 'in', 'the', 'moon']
indices: [0, 4]
result: ['is', 'in', 'the']
```